CM12002
Computer Systems Architectures

Data Representation

Fabio Nemetz

IEEE Standard for Binary Floating-Point Arithmetic

Institute of Electrical and Electronics Engineers

STD 754 (1985)

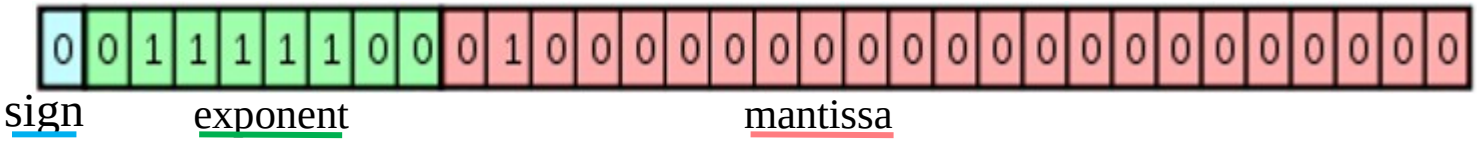
- How to encode binary floating-point numbers in a convenient manner
- Two basic formats:
  - Single precision: 4 bytes (32 bits)
  - Double precision: 8 bytes (64 bits)

IEEE 754 Single Precision (32 bits)

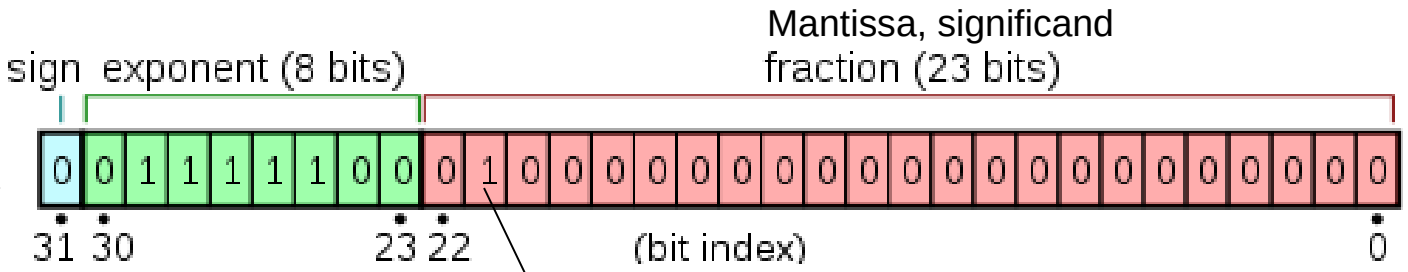
Let’s start with an example: if the following sequence of 32 0s and 1s is a single-precision floating-point number, then how do I convert it to decimal?

00111110001000000000000000000000

First, identify its 3 parts: **00111110001000000000000000000000**



IEEE 754 Single Precision (32 bits)



- Significand/mantissa precision: 24 (23 explicitly stored + 1, because the significand of a normalized binary floating-point number always has a 1 to the left of the binary point)
- Exponent: 0 to 255 (need to subtract *bias* which is 127)

(note: 0 and 255 are special cases)

$2^7 - 1 = 127$

From the example above, convert to decimal :

Sign:

Exponent:

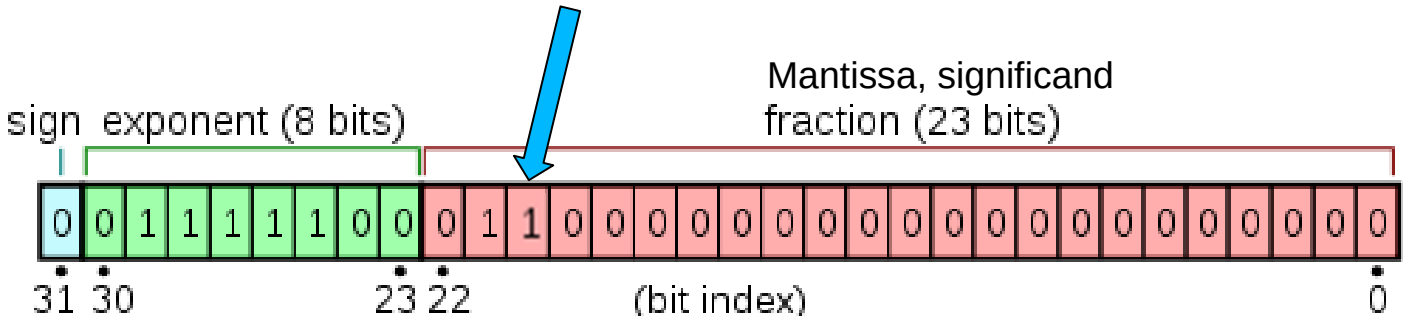
Mantissa (significand):

Value:

(127 is the bias (see above))

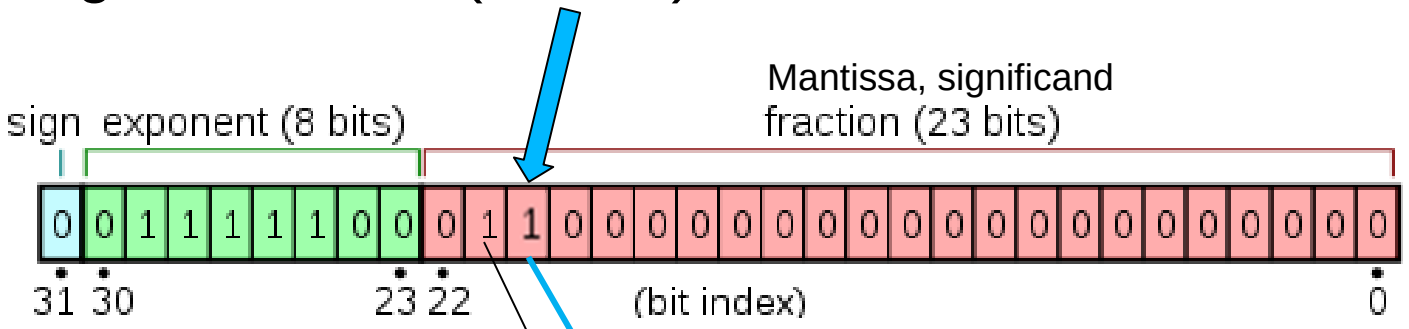
IEEE 754 Single Precision (32 bits)

What happens if I change this bit of the mantissa to 1???



Remember:
Binary fractions (similar to decimal fractions, but now base is 2):
$0.1 = (1/2) = 2^{-1}$
$0.01 = (1/4) = 2^{-2}$
$0.001 = (1/8) = 2^{-3}$

IEEE 754 Single Precision (32 bits)



- Significand/mantissa precision: 24 (23 explicitly stored + 1, because the significand of a normalized binary floating-point number always has a 1 to the left of the binary point)
- Exponent: 0 to 255 (need to subtract *bias* which is 127)

(note: 0 and 255 are special cases)

From the example above, convert to decimal :

Sign: 0 (positive)

Exponent: 01111100 = 124 → 124-127= -3

Mantissa (significand): $1+ 2^{-2} + 2^{-3}= 1+1/4 + 1/8= 1.375$

Value: $1.375 \times 2^{-3} = 1.375 \times 1/8 = 0.171875$

Practice:

Convert the IEE 754 single precision binary number:
1000 1000 1000 1000 1000 0000 0000 0000
into a decimal number

Try it on your own and then check it step-by-step here:
https://www.youtube.com/watch?v=4DfXdJdaNYs

IEEE 754 Single Precision (32 bits)

How to convert **13.1875** to single precision floating point number.

1) **The Sign**: it's positive, so sign=**0**
2) **Mantissa**:
   a) 13 = 1101_2
   b) 0.1875 keep multiplying by 2 until you reach the 0 (or you notice repetition).

0.1875 x 2 = 0.375 0.375 x 2 = 0.75 0.75 x 2 = 1.5 0.5 x 2 = 1

4) 1101.0011
5) To normalize to 1.1010011 → we need to move the binary point 3 times to the left
   - So it's 1.1010011 x $\mathbf{2^3}$ → First 1 is not stored, so: **1010011**
6) **Exponent** (add bias): 3 + 127 = 130 = 10000010

Result: 01000001010100110000000000000000

Obs: step 2b can be done in different ways

See explanation here: http://www.youtube.com/watch?v=MIrQtuoT5Ak

IEEE 754 Single Precision (32 bits)

Convert **173.7** to single precision floating point number.

1) **The Sign**:
2) **Mantissa**:
   a) 173 =
   b) 0.7 keep multiplying by 2 until you reach the 0 (or you notice repetition)

   0.7 x 2 =

4)
5) To normalize to → we need to move the binary point __ times to the __
   - So it's _____ x 2 → First 1 is not stored, so: _____
6) **Exponent** (add bias): __ + 127 = _____

Result:

Sign	Exponent	Mantissa

Suggestion to check exercises:
IEEE 754 Single Precision Converter: http://www.h-schmidt.net/FloatConverter/IEEE754.html

Practice:

Convert 45.45 to IEEE 754 single precision binary number format:

Try it on your own and then check it step-by-step here:
https://www.youtube.com/watch?v=tx-M_rqhuUA

Convert 263.3 to IEEE 754 single precision binary number format:
https://www.youtube.com/watch?v=8afbTaA-gOQ

IEEE 754 Single Precision (32 bits)

Exponent special cases: an exponent of 0 represents a zero-valued floating point number and an exponent of 255 represents infinity (if mantissa all zero)

The exponent values from 1 to 254 represent binary exponents from 2^{-126} through 2^{+127}. This is equivalent to a range of approximately 10^{-38} through 10^{+38}.

IEEE 754 Single Precision (32 bits)

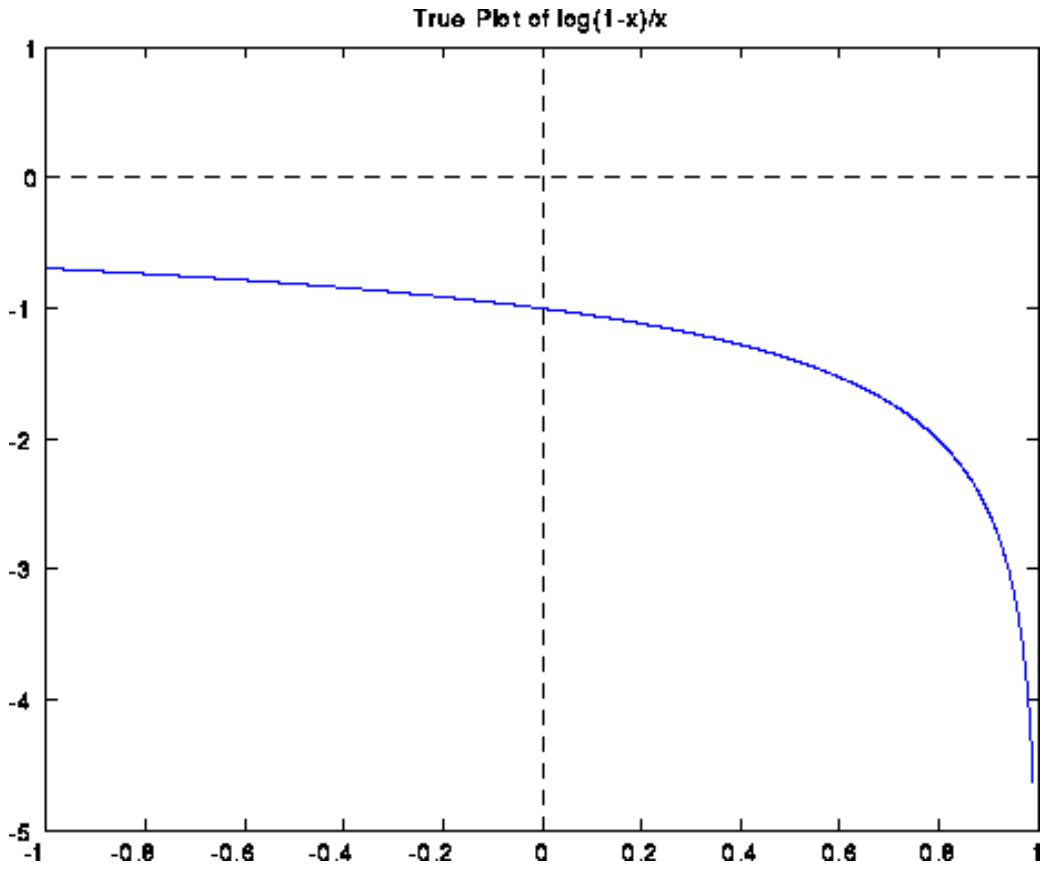
This is sufficient to represent e.g.

- Planck's constant: 6.626069×10^{-34} Js (quantum mechanics)
- Avogadro's constant: $6.022142 \times 10^{23} mol^{-1}$

to seven significant figures

Rounding Errors

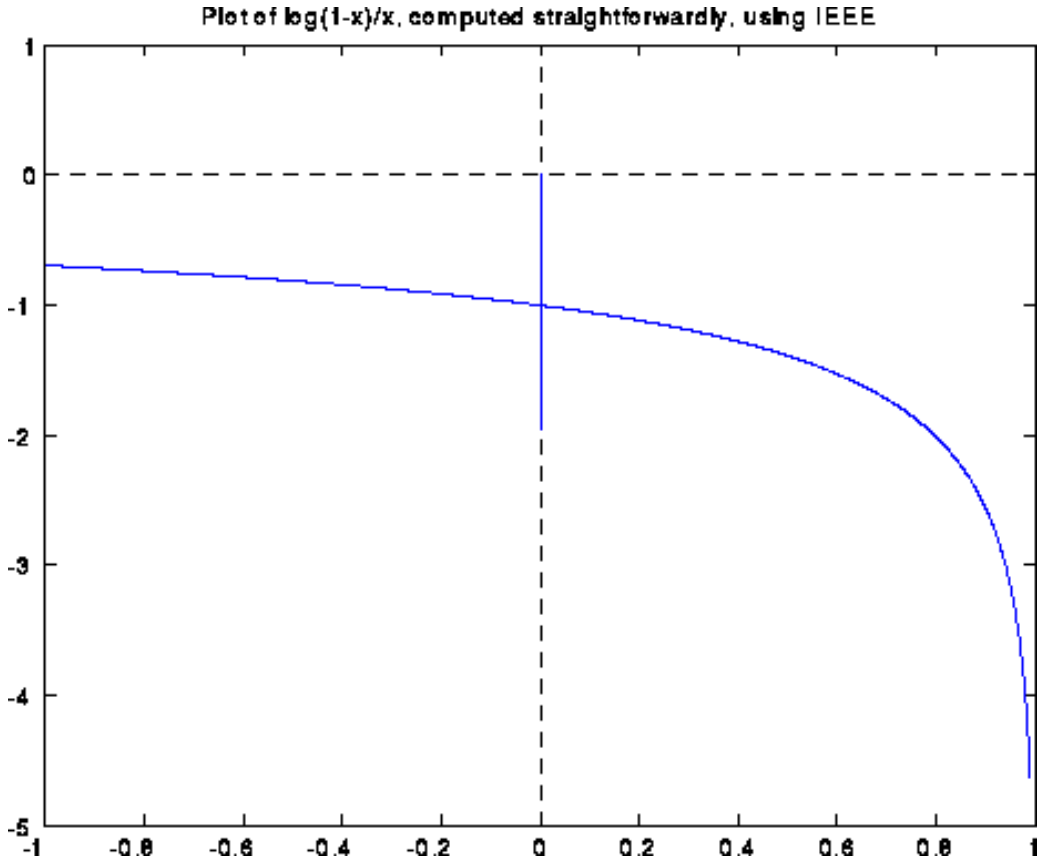
The function f(x) = log(1-x)/x is smooth for x <= 1.



Credit: http://www.cs.berkeley.edu/~demmel/cs267/lecture21/lecture21.html

Rounding Errors

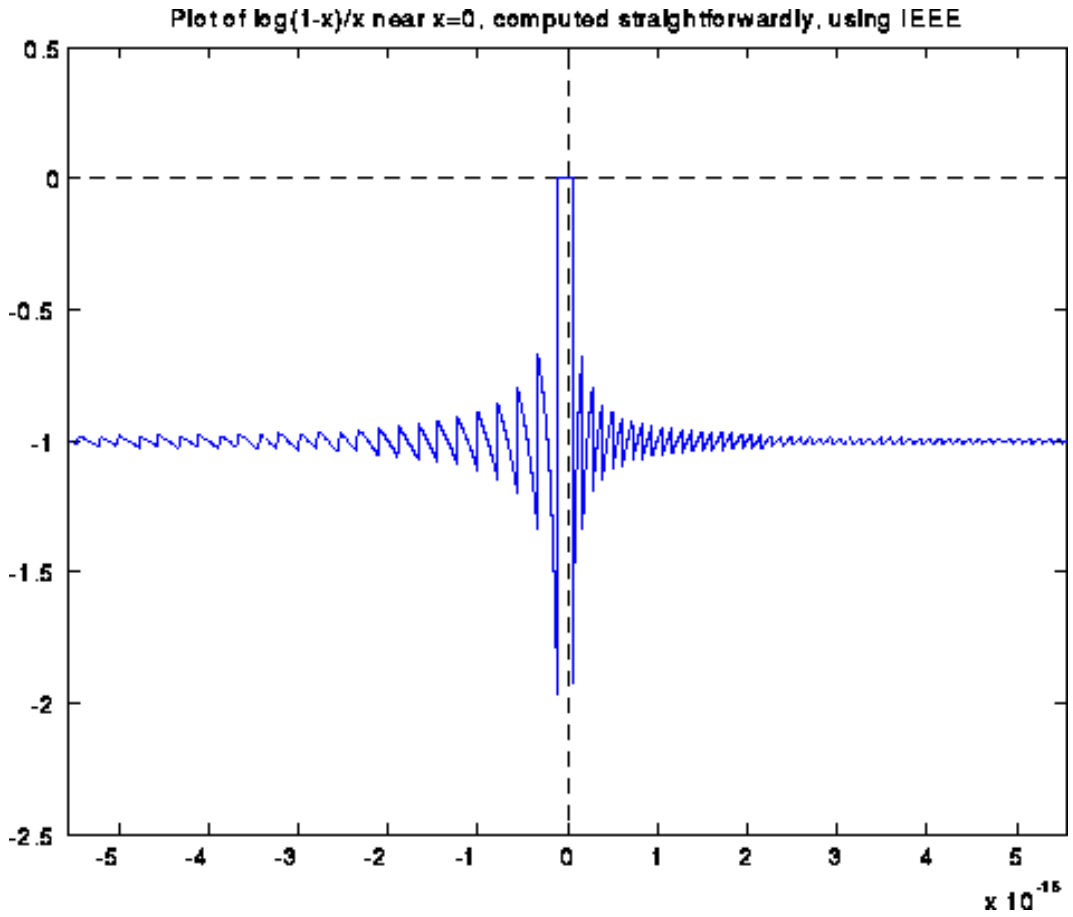
If we evaluate log(1-x)/x in IEEE floating point arithmetic using this formula, the computed graph is shown below. Note that the answer is quite wrong near x=0, with a spike going from -2 to 0:



Credit: http://www.cs.berkeley.edu/~demmel/cs267/lecture21/lecture21.html

Rounding Errors

Blowing up the graph near x=0 shows a sort of periodic behaviour:



Credit: http://www.cs.berkeley.edu/~demmel/cs267/lecture21/lecture21.html

Floating point arithmetic

Addition

Illustrated in decimal for convenience:

$(9.6 \times 10^2) + (6.6 \times 10^1)$

1) De-normalize the smaller operand and adjust its exponent to equal that of the other operand:

   $(9.60 \times 10^2) + (0.66 \times 10^2)$

2) Add the mantissae and retain the common exponent:

   (10.26×10^2)

3) Re-normalize the mantissa and adjust the exponent if necessary:

   (1.03×10^3)

Floating point arithmetic

Multiplication

Illustrated in decimal for convenience:

$(5.2 \times 10^{-4}) \times (2.6 \times 10^{7})$

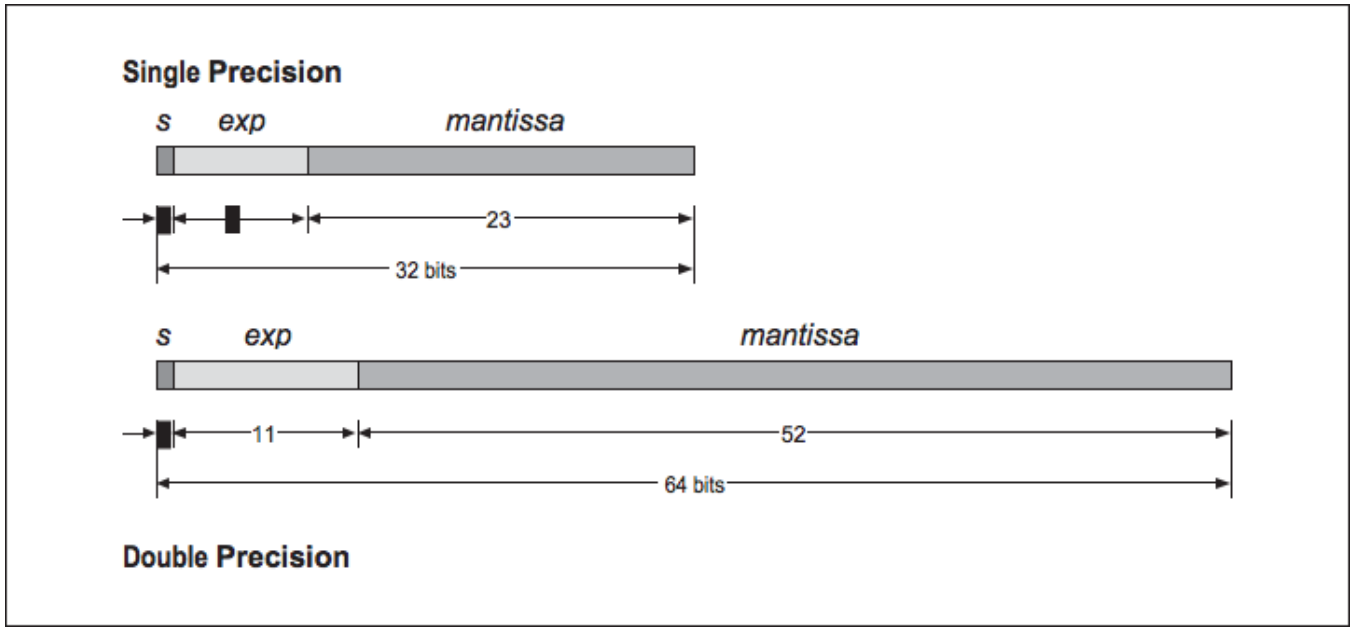
1) Multiply the mantissae and *add* the exponents. This will yield:

(13.56×10^{3})

2) Renormalize the mantissa and adjust the exponent if necessary:

(1.356×10^{4})

Double Precision



IEEE 754 – 2008 revision

Type	Sign	Exponent	Significand	Total bits	Exponent bias	Bits precision	Number of decimal digits
Half (IEEE 754-2008)	1	5	10	16	15	11	~3.3
Single	1	8	23	32	127	24	~7.2
Double	1	11	52	64	1023	53	~15.9
Double extended (80-bit)	1	15	64	80	16383	64	~19.2
Quad	1	15	112	128	16383	113	~34.0

Exercise

Convert IEEE 754 single precision floating point number:
01000010001010101100000000000000 to decimal:

01000010001010101100000000000000

0:
10000100:

01010101100000000000000:

Next time -

Representing and executing instructions

Answers to exercises:

Convert **173.7** to single precision floating point number
01000011001011011011001100110011 (see next slide)

Suggestion to check answers to exercises:
IEEE 754 Single Precision Converter:
http://www.h-schmidt.net/FloatConverter

Convert IEEE 754 single precision floating point number:
01000010001010101100000000000000 to decimal:

Answer: 42.6875

IEEE 754 Single Precision (32 bits)

Convert **173.7** to single precision floating point number.

1) **The Sign**: 0
2) **Mantissa**:
   a) $173 = 10101101_2$
   b) 0.7 keep multiplying by 2 until you reach the 0 (or you notice repetition)

   0.7 x 2 = **1**.4 0.4x2 =**0**.8 0.8x2=**1**.6 0.6x2=**1**.2 0.2x2=**0**.4 …(repeat 0110)

4) 10101101.1011001100110011
5) To normalize to 1.01011011011001100110011 → we need to move the binary point 7 times to the left
   - So it's $1.01011011011001100110011 \times 2^7$ → First 1 is not stored, so: 01011011011001100110011
   - 6) **Exponent** (add bias): 7 + 127 = 134 = 10000110

Sign | Exponent | Mantissa

Result: 0 10000110 01011011011001100110011

Suggestion to check exercises:
IEEE 754 Single Precision Converter: http://www.h-schmidt.net/FloatConverter/IEEE754.html